OOP and Java Basics

Object-Oriented Programming is programming with **classes** and **objects**.

A **class** is a template for creating **objects**. It defines variables and methods that all objects created from the class will have. Classes do not occupy memory until an object is created.

An **object** is an instance of a class with the attributes and methods defined by the **class**. Objects are stored in memory during runtime.

Java is an object-oriented programming language - everything in Java is an object. The Java Virtual Machine is a VM used to provide a runtime environment for Java bytecode, allowing the same compiled bytecode to run on different platforms through this VM.

```java
class Car{  4 usages  new *
    // Attributes/variables
    String model;  4 usages
    float speed = 0;  4 usages

    // Methods
    void go(){  2 usages  new *
        System.out.println("vroom vroom");
    }
    void accelerate(){  2 usages  new *
        this.speed++;
    }
    void stop(){  2 usages  new *
        this.speed=0;
    }
}
```

An example Car class with attributes and methods

```java
// the Main class is necessary for every Java program. MUST be public
public class Main {  new *
    // the main() method is the entrypoint of the program
    // MUST have this exact signature
    public static void main(String[] args) {  new *
        Car car1 = new Car(); // Object
        car1.model = "Ferrari";
        Car car2 = new Car(); // another Object of the same class
        car2.model = "Nissan Micra";
        car1.go();
        car2.go();
        for (int i=0; i<5; i++){
            car1.accelerate();
        }
        car2.accelerate();
        System.out.println("Speed of " + car1.model + ": " + car1.speed);
        System.out.println("Speed of " + car2.model + ": " + car2.speed);
        car1.stop();
        car2.stop();
    }
}
```

Example code creating and using two different *Car* objects.

All Java programs must have a *public class* containing a *public static void main(String[] args)* serving as the entry point into the program.

Java probably has the stupidest print statements out of any mainstream programming language - *System.out.println(…)* 🤮

Data Types in Java can be divided into **primitive** and **non-primitive**.

Primitive: byte, short, int, long, float, double, Boolean, char

Non-primitive: String, Array, Class

Basic syntax - if statements and loops:

```java
int four = 0;
while (true){
    if (four < 4){
        four++; // ++ -> +1 to the number
    } else if (four > 4){
        four--; // -- -> -1 from the number
    } else{
        break;
    }
}
```

```java
for (int k = 0; k < 10; k++) {
    if (k == 4) {
        continue; //stop this iteration and go to the next k
    }
    if (k == 8) {
        break; // exit the loop
    }
    System.out.println("k=" + k);
}
```

^ Outputs k = 1….8 excluding 4.

Classes and **Objects** can be created in Java as in the previous page. We can also add a **Constructor** to make instantiating objects easier. A **Constructor** is a public method in a class with the same name as the class, which allows us to initialise values of attributes or perform any initialisation computations.

```java
class Car{  4 usages  new *
    // Attributes/variables
    String model;  3 usages
    float speed = 0;  4 usages
    // Constructor
    public Car(String model){  2 usages
        this.model=model;
    }
```

```java
Car car1 = new Car( model: "Ferrari");
Car car2 = new Car( model: "Nissan Micra");
```

Using a Constructor we can now include initial values for the model at instantiation.

Classes can have 0, 1 or multiple constructors - must have different signatures.

We can use **Method Overloading** to define multiple **Constructors** with different signatures - upon instantiation, Java selects the correct one to use depending on input parameters.

```java
// Constructor 1: takes "model" parameter
public Car(String model){  2 usages  new *
    this.model=model;
}
// Constructor 2: no params, initialises to default value
public Car(){  1 usage  new *
    this.model="Unknown";
}
```

Note: any method can be overloaded!

Note: "*this*" refers to the current object, in this case the object being instantiated.

Important distinction - **Object** vs **Reference**:

An **Object** is the block of memory storing all the attributes and methods relating to an instance of a class. When we instantiate an object in Java, we define a variable which is a **Reference** to this block of memory.

```
Cow cow1 = new Cow();
```

Creates a variable that references to a Cow object

Creates the actual object using the class constructor (e.g. using computer memory to create class variables)

Together, this line of code creates the object and links it to the reference variable

This means that if we defined a second *Cow cow2 = cow1* we are only creating a second **Reference** to the same **Object** that *cow1* is referencing.

Statics - Class-Level Attributes and Methods:

Normal methods and attributes in Java belong to **Objects** - each **Object** gets its own "copy" of each (i.e. two different *Car* objects each have their own *model* variable).

Using the ***static*** keyword, we can define variables and methods shared between all objects of a class.

```
class Thing{  5 usages
    static int staticNumber = 0;  1 usage
    int normalNumber = 0;  4 usages
}

public class Main2 {
    public static void main(String[] args) {
        Thing dingus = new Thing();
        Thing mongoose = new Thing();
        Thing.staticNumber++; // accessed via class "Thing" directly
        dingus.normalNumber++; // accessed via object reference
        mongoose.normalNumber--;
        System.out.println(dingus.normalNumber);
        System.out.println(mongoose.normalNumber);
    }
}
```

Statics can only be accessed or modified via the class directly (*Thing.staticNumber* not *dingus.staticNumber*), unlike normal attributes and methods which are accessed via a specific object reference.

Note: Statics are memory-efficient since they are not recreated per object.

We can combine the ***static*** keyword with the ***final*** keyword to define **Constants**:

```java
public class Main2 {
    public static final String PENIS = "8======D";
    public static void main(String[] args) {
        System.out.println(PENIS);
    }
}
```

final variables cannot be modified after initialisation. Combining with ***static*** creates a global constant.

Static Methods also exist. These are class-specific methods that cannot access non-static variables or methods (as these are object-specific and a class can have any number or no objects).

```java
class Util{  1 usage
    public static final String PENIS = "8======D";
    // Static method (peak maturity right here)
    public static void printPenis(){  1 usage
        System.out.println(PENIS);
    }
}
public class Main2 {
    public static void main(String[] args) {
        Util.printPenis();
    }
}
```

Note: Since **static**s belong to classes, they can be used even if no objects of their class have been instantiated.

To check whether an Object is of a given Type, the ***instanceof*** operator can be used. This returns true or false depending if the Object is of the supplied type.

This extends to determining whether an Object is an instance of some Class - not just Java built-in Types.

```java
public static void main(String[] args) {  new *
    String penis = "8=======D";
    Car vauxhall = new Car();
    System.out.println(penis instanceof String);
    // Output: True
    System.out.println(vauxhall instanceof Car);
    // Output: True
    System.out.println(vauxhall instanceof String);
    // Output: False
```

Note: ***instanceof*** only works with Primitive Datatypes in Java 23 or newer.

Note: if the compiler detects that an ***instanceof*** will always return False, it will throw an error at compile-time.

OOP Principles in Java

The four key principles of Object Oriented Programming are **Encapsulation**, **Inheritance**, **Abstraction** and **Polymorphism**. Those are four big scary words…

Encapsulation: the bundling of data (**attributes**) and **methods** that operate on the data into a single unit (**class**), while restricting direct access to some of the object's components.

This protects data from erroneous modification, enhances security and data hiding and promotes modular, more maintainable code.

In Java, **encapsulation** is achieved through **access modifiers**:

- ***public***: accessible from anywhere in the program
- ***private***: accessible only from within the same class
- ***protected***: accessible from the same package, the same class or from subclasses (see **Inheritance**).

These modifiers can be added to any variable, method or class.

The main W from doing this is that access to attributes can be controlled, for example by making the attributes themselves ***private*** but creating ***public*** getter and setter methods that must be used to access or modify the attribute. Setter methods can also include validation to prevent attributes from being updated to erroneous values.

```java
class Pig{  2 usages
    private int age;  3 usages
    // Constructor
    public Pig(int age){  1 usage
        // validation - ensuring age isn't negative
        if (age < 0){
            age = 0;
        }
        this.age = age;
    }
    // Methods
    public void setAge(int age){  1 usage
        // validation
        if (age > 0){
            this.age = age;
        }
    }
    public int getAge(){  1 usage
        return this.age;
    }
}
```

```java
// Main
public class Main2 {
    public static void main(String[] args) {
        Pig pig = new Pig( age: 2);
        pig.setAge(-67);
        System.out.println(pig.getAge());
    }
}
```

Output: 2

Without encapsulation there would be no way of preventing attributes from being updated to erroneous values. (defensive programming)

Inheritance: A **subclass** (child class) can be derived from a **superclass** (parent class) to retain and update the variables and methods of the parent class.

If multiple classes share some logic (Methods and Attributes), we can reduce the code duplication by moving this shared logic to a **superclass** and having the existing classes be **subclasses** of this **superclass**. **Subclasses** inherit all *public* and *protected* methods and attributes of the **superclass**, and can also have their own class specific methods and attributes.

```
class Vehicle{  2 usages  2 inheritors
    public String model;  2 usages
    public int numWheels;  2 usages
}
class Bicycle extends Vehicle{  no usages
    // bicycle constructor
    public Bicycle(String model){  no usages
        this.numWheels = 2;
        this.model=model;
    }
}
class Bus extends Vehicle{  no usages
    // additional subclass attributes
    public int passengerCapacity;  1 usage
    public boolean isDoubleDecker;  1 usage
    // bus constructor
    public Bus(String model, int passengerCapacity, boolean isDoubleDecker){
        this.model=model;
        this.numWheels =4;
        this.passengerCapacity=passengerCapacity;
        this.isDoubleDecker=isDoubleDecker;
    }
}
```

As a toy example, we can define a shared superclass "*Vehicle*" which includes logic shared by all vehicles, and then each vehicle type can have its own subclass defining specific logic for that type of vehicle.

Note: In Java, use the ***extends*** keyword in the subclass definitions.

We can use **Method Overriding** to redefine a superclass' method in a subclass, e.g. to adapt it specifically to the subclass. To do this in Java, use ***@Override***.

```
class Animal {  2 usages  2 inheritors
    void makeSound(){  no usages  1 override
        System.out.println("*animal noise*");
    }
}
class Pig extends Animal {  no usages
    @Override void makeSound(){  no usages
        System.out.println("oink");
    }
}
class Bee extends Animal {  no usages
    @Override void makeSound(){  no usages
        System.out.println("bzzzzzzzzzzzzzzzzzz");
    }
}
```

```
// Main
public class Main2 {
    public static void main(String[] args) {
        Animal animal = new Animal();
        Pig pig = new Pig();
        Bee biswas = new Bee();
        animal.makeSound();
        // >> *animal noise*
        pig.makeSound();
        // >> oink
        biswas.makeSound();
        // >> bzzzzzzzzzzzzzzzzzz

    }
}
```

Note: this is different to **Method Overloading**.

We can also call superclass methods from a subclass using the **super** keyword.

```java
class SuperClass{  2 usages  1 inheritor
    public void doThing(){  1 usage  1 override
        System.out.println("Superclass method");
    }
}
class SubClass extends SuperClass{  no usages
    @Override public void doThing(){  1 usage
        System.out.println("Override method");
    }
}
class SubClass2 extends SuperClass{  no usages
    @Override public void doThing(){  1 usage
        System.out.println("Override method with super");
        super.doThing();
        // >> Superclass method
    }
}
```

Output of *SuperClass2.doThing()* :

>> Override method with super

>> Superclass method

Note: this is a stupid example to demonstrate that **super** is a thing that exists.

Abstraction: A **superclass** designs the inputs and outputs of methods, leaving the implementation of the actual logic to the **subclass**.

Abstraction allows a **superclass** to define the high-level structure (i.e. method signatures) while leaving the actual specific implementation to **subclasses**. This ensures that classes can share a common **interface** without revealing implementation details. This can help reduce complexity, enhance reusability and improve maintainability.

E.g. a programmer working with a class written by someone else can see what it does by looking at its **interface**, and doesn't need to know how that is implemented.

To code with Abstraction in Java, use **Abstract Classes** and **Interfaces**.

Abstract Classes are classes declared with the ***abstract*** keyword. These classes cannot be instantiated directly, some other **concrete** (non-abstract) class must inherit from them. Abstract Classes can contain both **abstract** (no body) and **concrete** (with body) methods, and variables work as normal, but need to use ***protected*** in place of ***private*** since we need them to be accessible by concrete subclasses to be able to work with them.

Note: Java will yell at you if you do not provide an implementation for all abstract methods in a concrete subclass.

```java
abstract class Vehicle{  1 usage  1 inheritor
    // note use of "protected" instead of "private" here
    protected String model;  1 usage
    protected int speed = 0;  3 usages
    // abstract method - MUST be defined in all subclasses
    abstract void drive();  1 usage  1 implementation
    // concrete methods
    void setSpeed(int newSpeed){  1 usage
        this.speed = newSpeed;
    }
    int getSpeed(){  no usages
        return this.speed;
    }
    void printSpeed(){  1 usage
        System.out.println(this.speed);
    }
}
```

Example: concrete class *Car* inherits from abstract class *Vehicle*.

```java
// concrete class Car inheriting from Vehicle
class Car extends Vehicle{  2 usages
    public Car(String model){  1 usage
        this.model = model;
    }
    public void drive(){  1 usage
        System.out.println("vroom vroom");
    }
}

// Main
public class Main {
    public static void main(String[] args) {
        Car car = new Car( model: "Nissan Micra");
        car.drive(); // Output: vroom vroom

        // concrete methods defined in abstract Vehicle class
        car.setSpeed(67);
        car.printSpeed(); // Output: 67
    }
}
```

Interfaces are similar, but <u>all</u> non-static methods have to be ***abstract*** or ***default***. Variables can only be ***public***, ***static*** and ***final***. ***static*** methods of interfaces are <u>not</u> inherited.

Unlike **Abstract Classes**, **Interfaces** allow for **Multi-Inheritance** - a single class can implement multiple Interfaces.

Note: need to use ***implements*** instead of ***extends*** when inheriting from **Interfaces**.

```java
// Interfaces
interface CanFly {  1 implementation
    // interface methods are public and abstract by default
    // the keywords are unnecessary
    void fly();  1 implementation
    int getAltitude();  1 implementation
    void setAltitude(int altitude);  1 implementation
}
interface Vehicle {  1 implementation
    void accelerate();
    void decelerate();
    int getSpeed();
}
```

Example: concrete class *Aircraft* implements <u>both</u> *CanFly* and *Vehicle* interfaces - it needs to provide an implementation for <u>all</u> methods defined across both interfaces.

Note: a class can implement more than 2 interfaces if needed.

```java
// Multi-Inheritance: an aircraft can fly and is a vehicle
class Aircraft implements CanFly, Vehicle {
    private int altitude;
    private int speed = 0;
    public String model;
    // Constructor
    public Aircraft(String model){
        this.model = model;
        this.altitude = 0;
    }
    // Methods
    // Needs to implement all abstract methods from all Interfaces it implements
    public void accelerate(){
        this.speed++;
    }
    public void accelerate(int amount){  // cheeky method overloading ;)
        this.speed+= amount;
    }
    public void decelerate(){
        this.speed--;
    }
    public int getSpeed(){
        return this.speed;
    }
    public void fly(){
        System.out.println(this.model + " is flying");
    }
    public int getAltitude(){
        return this.altitude;
    }
    public void setAltitude(int altitude){
        this.altitude = altitude;
    }
}
```

We can also use **Interfaces** as types in function parameters and the function will accept any object of a class implementing the Interface.

(We can also use normal/abstract Classes in this way.)

```java
interface Building{  2 usages  1 implementation
    int getHeight();  1 usage  1 implementation
}

class Skyscraper implements Building{  2 usages
    private int height;  2 usages
    public Skyscraper(int height){  1 usage
        this.height = height;
    }
    public int getHeight(){  1 usage
        return this.height;
    }
}
```

With this *Building* class and the *Aircraft* class from before, we can have a function that takes any *Building* and any *Aircraft*.

Here, *Aircraft* is a Class and *Building* is an Interface.

```java
public class Main {
    /*
    We can use Interfaces as types in functions and all implementing classes can
    be passed in.
    E.g. "Building building" will accept any object of any class that implements
    the Building interface
    */
    private static void nineEleven(Building building, Aircraft aircraft){  1 usage
        // building.getHeight() calls the relevant implementation of getHeight()
        // in this case, the one in the Skyscraper class.
        if (building.getHeight() < aircraft.getAltitude()){
            System.out.println("all chill");
        } else{
            System.out.println("kaboom");
        }
    }
    // Entry point
    public static void main(String[] args) {
        Skyscraper TwinTowers = new Skyscraper( height: 417);
        Aircraft Boeing767 = new Aircraft( model: "Boeing 767");
        Boeing767.accelerate( amount: 100);
        Boeing767.fly(); // Output: Boeing 767 is flying
        Boeing767.setAltitude(200);
        nineEleven(TwinTowers, Boeing767); // Output: kaboom
    }
}
```

Important Distinction - **Abstract Classes** vs **Interfaces**:

Feature	Abstract Class	Interface
Method Implementation	Can have both abstract and fully implemented methods	Can have abstract methods, **default methods** (with implementation), and **static methods** (with implementation)
Constructors	**Can have** constructors	**Cannot** have constructors
Instance Variables	Can have instance variables	Only **static** and **final** constants (by default)
Multiple Inheritance Support	**No** multiple inheritance (can only extend one class)	**Yes**, a class can implement multiple interfaces
Access Modifiers	Can have `public`, `protected`, `private`, or `default` methods	Methods are **public** by default (but Java 9 added `private` methods for internal reuse)
Use Case	Used when different classes share common behavior and fields	Used for defining contracts that multiple classes can implement

Use **Abstract Classes** when we need to share variables and methods among related classes.

Use **Interfaces** if multiple inheritance is needed.

Polymorphism: Objects and methods have "many forms" – Objects can be treated as different classes. Methods can be overridden or hidden.

It is an overarching term for various programming techniques in Java.

Compile-Time Polymorphism: **Method Overloading**

This is where multiple methods have the same name but with different signatures. It is determined at compile-time which version of the method is called based on the passed parameters.

```
public void accelerate(){  no usages
    this.speed++;
}
public void accelerate(int amount){
    this.speed+= amount;
}
```

The method *accelerate()* has "many forms" (well, two forms)

Method Overloading always happens within the same class.

Run-Time Polymorphism: **Method Overriding**

This is where a Subclass provides a specific implementation of a Superclass method. It is determined at runtime which version of the method is used.

```
class Car{  3 usages  1 inheritor  new *
    void start(){  2 usages  1 override  new *
        System.out.println("Engine started");
    }
}
class ElectricCar extends Car{  2 usages  new *
    @Override  2 usages  new *
    void start(){
        System.out.println("Electric motor started");
    }
}
```

The method *start()* has "many forms".

Method Overriding always happens with Superclasses and Subclasses and so requires Inheritance, unlike **Method Overloading**.

Upcasting

This is where a Subclass object is assigned to a Superclass reference. This object would be able to access all non-***private*** attributes/methods of the superclass, and overridden methods in the Subclass. However, methods and attributes that exist only in the Subclass would not be accessible.

This feature of Java is what allows methods that take Superclass objects as parameters to also accept Subclass objects (which will then be internally upcasted). It also allows **Collections** (e.g. Arrays) of type *Superclass* to be able to contain Objects of any Subclass of *Superclass*.

```java
class Car{  6 usages  1 inheritor  new *
    void start() {  2 usages  1 override  new *
        System.out.println("Engine started");
    }
}
class ElectricCar extends Car{  2 usages  new *
    @Override  2 usages  new *
    void start(){
        System.out.println("Electric motor started");
    }
    void charge(){  no usages  new *
        System.out.println("Electric car is charging");
    }
}
```

```java
public static void main(String[] args) {  new *
    Car car = new Car();
    // Implicit Upcasting
    Car gwiz = new ElectricCar();
    // Explicit Upcasting
    Car tesla_cyberfuck = (Car) new ElectricCar();
    car.start(); // Output: Engine started
    gwiz.start(); // Output: Electric motor started
    tesla_cyberfuck.charge(); // Error: no charge() in Car class
    // (can only use methods defined in superclass)
}
```

Implicit vs Explicit Upcasting - logically they are the same.

```java
public static void main(String[] args) {  new *
    Car volvo = new Car();
    ElectricCar tesla_cyberfuck = new ElectricCar();
    Car[] garage = {volvo, tesla_cyberfuck};
    for (int i = 0; i < garage.length; i++){
        garage[i].start(); // calls the relevant form of "start()"
        garage[i].charge() // Error: class Car has no method charge()
    }
}
```

All methods called on items in an Array need to exist for the type of the Array (in this case, *Car*).

Note: the above example would still fail if all *Car*s in the garage are *ElectricCar*s, since all items get automatically **upcasted** to the type of the Array (i.e. *Car*).

Downcasting

This is where a superclass reference is typecasted to a subclass reference. *This only works if the object being referenced was first initialised as type Subclass*, and must be done explicitly.

```java
public static void main(String[] args) {  new *
    // Upcast
    Car tesla_cyberfuck = new ElectricCar();
    // Downcast
    ElectricCar e = (ElectricCar) tesla_cyberfuck;
    // Works:
    e.charge();

    // Doesn't Work:
    Car volvo = new Car();
    ElectricCar electricVolvo = (ElectricCar) volvo;
    // Error: class Car cannot be cast to class ElectricCar
}
```

To downcast a reference, the object being referenced must have been initially declared as being of the type being downcasted to.

A *Car* cannot magically become an *ElectricCar.*

It is recommended to use ***instanceof*** before downcasting to check whether the object can be safely downcasted (i.e. without the program crashing at runtime).

Note: if the compiler detects that an ***instanceof*** will return False, it will throw an error at compile-time.

```java
public static void main(String[] args) {  new *
    // Define three Cars, 2 of which are ElectricCars
    ElectricCar tesla_cyberfuck = new ElectricCar();
    Car volvo = new Car();
    ElectricCar gWiz = new ElectricCar();
    // All members of garage are upcasted to type Car
    Car[] garage = {tesla_cyberfuck, volvo, gWiz};
    for (int i = 0; i < garage.length; i++){
        // Check if each Car Object is an instance of ElectricCar
        if (garage[i] instanceof ElectricCar){
            // If it is, downcast to ElectricCar and call charge()
            ((ElectricCar) garage[i]).charge();
        }
    }
    // Output: Electric car is charging (x2)
}
```

Type checking before downcasting ensures that the program won't crash.

Downcasting should generally be avoided unless necessary.

Java Collections

The example at the end of the previous section uses an Array *garage* of type *Car[]*. Arrays defined in this way have a fixed length and have no built-in methods for searching or sorting.

We can use **Collections** (defined as part of the Java Collections Framework), which are extensions of arrays that provide pre-built, reusable, dynamic solutions to managing groups of objects in Java. These collections allow for dynamic resizing, have built-in methods and can support different data types using **Generics**. A basic example:

```java
public static void main(String[] args) {  new
    // Dynamic list that accepts any Objects.
    List<Object> list = new ArrayList<>();
    Number num = 5;
    String penis = "8===D";
    list.add(num);
    list.add(penis);
    System.out.println(list);
    // Output: [5, 8===D]
}
```

<Object> is a type generic. A list can have multiple generics to specify multiple allowed types. This will implicitly upcast all members of the *List* to type *Object*, so only methods of class *Object* can be applied to list members.

Sidequest: **Generics** can also allow us to use Types as parameters to methods or classes, allowing us to handle multiple datatypes from a single function or class.

For example:

```java
// Bounded Generic: accepts anything of type Number,
// or an instance of any subclass of Number (Integer, Double etc)
public static <T extends Number> double add(T a, T b){  1 usage  new *
    return  a.doubleValue() + b.doubleValue();
    // doubleValue() is a method of the Number class that coverts it
    // to a double. We can use any method from the Number class since
    // the inputs, which are of type T, must extend Number.
}
public static void main(String[] args) {  new *
    System.out.println(add( a: 3,  b: 3.7));
    // Output: 6.7
}
```

We can use any class we want in Generics.

Note: **primitives** do not work here as they are not Objects.

Back to Collections: The Java Collections Framework is divided into four categories:

- **List**: Ordered collection, allows duplicates
- **Set**: Unique elements, no duplicates
- **Map**: Key-Value pairs, no duplicate keys
- **Queue**: FIFO structure

Each of these categories contains multiple implementations (e.g. **Queue** contains **Queue** and **PriorityQueue**.)

List:

An **ArrayList**: works like a regular Array, but will resize if needed.

```java
public static void main(String[] args) {  new *
    // Creating a new ArrayList
    ArrayList<String> morons = new ArrayList<>();

    // adding elements to an ArrayList
    morons.add("adolf");
    morons.add("nigel");
    // inserting at a specific position
    morons.add( index: 1,  element: "donald");
    morons.add("harry");
    // outputting entire list
    System.out.println(morons);
    // Output: [adolf, donald, nigel, harry]

    // removing elements from an ArrayList
    // remove by name
    morons.remove( o: "adolf");
    // or by index
    morons.remove( index: 2);

    // accessing specific element
    System.out.println(morons.get(1));
    // Output: nigel
}
```

List Operations:

Elements can be added using *.add()* - optionally with a specific index at which the element is to be inserted

Elements can be removed by using *.remove()* - with either a specific index or a specific item to remove.

Elements can be accessed by index by using *.get()*

LinkedList: Internally, a doubly linked list. Functionally, works like ArrayList, with the exact same syntax for *get()*, *add()* and *remove()*.

Map:

The only type of Map we need to know about is the **HashMap**. This works similarly to a Python Dictionary, storing key-value pairs such that keys are unique. A **HashMap** uses hashing for fast key lookups.

```java
public static void main(String[] args) {  new *
    // Create a HashMap with keys of type String and values of type Integer
    HashMap<String, Integer> highScores = new HashMap<>();

    // Add key/value pairs to the HashMap
    highScores.put("Jonathan", 9000);
    highScores.put("James", 4);
    highScores.put("Jack", 6700);
    // Duplicates get overwritten
    highScores.put("Jack", 6900);

    // Remove key/value pairs from the HashMap - by key since keys are unique
    highScores.remove( key: "James");

    // Output HashMap
    System.out.println(highScores);
    // Output: {Jonathan=9000, Jack=6900}

    // Get specific value
    System.out.println(highScores.get("Jonathan"));
    // Output: 9000
}
```

HashMap Operations:

Create/update key/value pairs with *.put(key, value)*

Remove key/value pairs with *.remove(key)*

Get a value corresponding to a key with *.get(key)*

Set:

The only Set implementation we need to know about is the **HashSet**. This is built on top of a HashMap but only stores unique values and does not preserve order.

Operations:

.add() and *.remove()* as with Lists, but cannot use indexes as a Set is unordered

.contains() to check whether an element is in the HashSet - Note: all Collections have this method.

```java
public static void main(String[] args) {  new *
    // Create a HashSet
    HashSet<String> morons = new HashSet<>();

    // Add elements to the HashSet
    morons.add("adolf");
    morons.add("donald");
    morons.add("nigel");
    // Duplicates get ignored
    morons.add("adolf");

    // Remove elements from the HashSet
    morons.remove( o: "nigel");

    // Output the HashSet
    System.out.println(morons);
    // Output: [adolf, donald]

    // Check whether an element is in the HashSet
    System.out.println(morons.contains("adolf"));
    // Output: true
}
```

Performance Comparisons:

Operation	Array	ArrayList	LinkedList	HashMap
Access (get)	O(1) ✅	O(1) ✅	O(n) ❌	O(1) ✅
Insert (add)	O(n) ❌	O(1) ✅	O(1) ✅	O(1) ✅
Delete (remove)	O(n) ❌	O(n) ❌	O(1) ✅	O(1) ✅

ArrayList → Best for fast lookups, bad for insert/remove
LinkedList → Best for frequent insertions/removals
HashMap → Best for key-value mapping & fast lookups
HashSet → Best for unique element storage
Avoid using LinkedList for large lists unless frequent modifications are needed

Iterators: looping through Collections

We can use for loops to iterate through some Java Collections, but they do not safely handle the removal of elements from within the loop. Instead, use **Iterator**s.

```
public static void main(String[] args) {  new *
    HashMap<String, Integer> highScores = new HashMap<>();

    highScores.put("Jonathan", 9000);
    highScores.put("James", 4);
    highScores.put("Jack", 6700);
    highScores.put("Jimbob", 5001);

    // Create an Iterator for the HashMap values
    Iterator<Integer> it = highScores.values().iterator();
    // Safe way of iterating through a HashMap with deletions
    // Remove all key/value pairs where the value < 6000
    while(it.hasNext()){
        Integer score = it.next();
        if (score < 6000){
            it.remove();
        }
    }
    System.out.println(highScores);
}
```

Iterators can be defined for any Java Collection type.

Operations:

.hasNext() - returns *true* or *false* depending on whether there is a next element

.next() - returns the next value

.remove() - removes the current value from the Collection.

Collections Notes:

If we add an Object to a Collection, we actually only pass the Reference to the collection -> any modifications made to the Object in the collection will also apply to the Object outside of the collection as they are the same Object.

To avoid this, we need to clone the object. This requires defining a **Copy**

Constructor in the class - Note: there is no default, this has to be written manually for each class.

```
class Skyscraper implements Building{  3 u
    // Attributes
    private int height;  4 usages
    // Normal Constructor
    public Skyscraper(int height){  1 usage
        this.height = height;
    }
    // Copy Constructor
    public Skyscraper(Skyscraper other){
        this.height = other.height;
    }
    // Methods
    public int getHeight(){  1 usage
        return this.height;
    }
}
```

A **Copy Constructor** takes another Object of the same Class as argument and specifies how to copy the data from this other object to a new Object, producing a clone of the object.

Use this when shared references are not wanted (*Building2 = Building1* just creates a second reference to the same object)

If an Object Reference doesn't actually point to any Object, it is **null**. It is best practice to always check for **null** before attempting to access an object.

All Java Collections have built-in methods *.sort()* and *.contains()* which should be used for sorting and searching, instead of reinventing the ferris wheel.

OOP Design Patterns

OOP Design Patterns are reusable, efficient solutions to common problems in OOP software design. They typically fall under three categories: **Creational** - relating to object creation, **Structural** - relating to class relationships, and **Behavioural** - relating to object interactions.

There are many patterns in each category, but this course only focusses on the **Singleton** (Creational), **Factory** (Creational) and **Observer** (Behavioural).

Singleton: A pattern which ensures that a class has only one instance at any time, providing a global controlled access to that instance. This is useful when we need something that must not be instantiated multiple times, e.g. collections of global variables, thread pool managers and thread synchronisation (locks etc).

Example implementation:

```java
class Singleton {  5 usages  new *
    // Stores the single instance that can exist
    // Static -> can be only one
    private static Singleton instance;  3 usages
    // Private constructor - cannot be instantiated by an external class
    private Singleton(){}  1 usage  new *
    // Public method to get the instance
    // Define a new instance if it doesn't exist, otherwise return the instance
    public static Singleton getInstance(){  1 usage  new *
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
    // Methods
    public void doShit(){  1 usage  new *
        System.out.println("shit is being done");
    }
}
```

```java
public class Main {  new *
    public static void main(String[] args) {  new *
        Singleton singleton = Singleton.getInstance();
        singleton.doShit();
        // Output: shit is being done
    }
}
```

The **Singleton** instance is accessed solely through the *getInstance()* static method.

Note: Singletons may introduce unnecessary dependencies and global states.

Abstract Factory: A pattern where object creation is delegated to a factory object, instead of creating objects directly. This factory can decide at runtime which subclass of object to return. Use of factories encapsulates object creation and factories are easily extensible if new subclasses are added.

Example:

```java
class Shape {  7 usages  2 inheritors  new *
    public void announce(){  3 usages  2 overrides
        System.out.println("I am a Shape");
    };
}
class Circle extends Shape{  1 usage  new *
    @Override  3 usages  new *
    public void announce(){
        System.out.println("I am a Circle");
    }
}
class Rectangle extends Shape{  1 usage  new *
    @Override  3 usages  new *
    public void announce(){
        System.out.println("I am a Rectangle");
    }
}
```

```java
class ShapeFactory{  3 usages  new *
    public static Shape getShape(String species){  3 usages  new *
        switch (species){
            case "CIRCLE":
                return new Circle();
            case "RECTANGLE":
                return new Rectangle();
        }
        return new Shape();
    }
}
public class Main {  new *
    public static void main(String[] args) {  new *
        Shape shape1 = ShapeFactory.getShape( species: "RECTANGLE");
        Shape shape2 = ShapeFactory.getShape( species: "CIRCLE");
        Shape shape3 = ShapeFactory.getShape( species: "");
        shape1.announce(); // Output: I am a Rectangle
        shape2.announce(); // Output: I am a Circle
        shape3.announce(); // Output: I am a Shape
    }
}
```

Note: there is also the Factory Method pattern, which replaces direct object construction calls with a Factory Method in the superclass (or interface), which can then be overridden by subclasses according to their needs.

Observer: defines a one-to-many dependency between one **Subject** and multiple **Observer**s. The **Subject** maintains a list of **Observers**. When the state of the **Subject** changes, all **Observers** are notified. This is useful for handling events, e.g. interactions with the GUI.

Example implementation:

```java
// Subject class
class Subject{  2 usages  new *
    // Observer list
    private List<Observer> observers = new ArrayList<>();
    // Method to add an Observer to the list
    public void addObserver(Observer observer){  2 usages
        observers.add(observer);
    }
    public void notifyObservers(String msg){  1 usage  new *
        Iterator<Observer> it = observers.iterator();
        while (it.hasNext()){
            Observer o = it.next();
            o.update(msg);
        }
    }
}

// Observer interface
interface Observer{  5 usages  1 implementation  new *
    void update(String msg);  1 usage  1 implementation  new *
}
// concrete Observer class
class User implements Observer{  4 usages  new *
    @Override  1 usage  new *
    public void update(String msg){
        System.out.println("Received message: " + msg);
    }
}
```

```java
// Main
public class Main {  new *
    public static void main(String[] args) {  new *
        // Initialise Subject and Observers
        Subject broadcaster = new Subject();
        User user1 = new User();
        User user2 = new User();
        // add the Observers to the Subject's list
        broadcaster.addObserver(user1);
        broadcaster.addObserver(user2);
        // Send a message to the Observers
        broadcaster.notifyObservers( msg: "you are an idiot");
        // Output: you are an idiot (x2)
    }
}
```

The **Subject** sends "messages" to all **Observers** in its list whenever a state is changed. The **Observers** listen for messages and perform actions when it receives them. Different messages could correspond to different state changes and different actions.

UML Class Diagrams

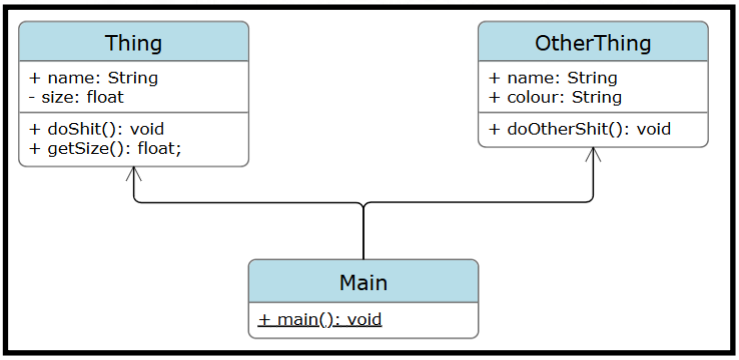
UML Diagrams are a standard way of visualising classes and their relationships. For each class, we include the names and types of all attributes and methods, with **static** methods being underlined.

We signify access modifiers with:

+ => public

- => private

=> protected



Arrows are used to signify different types of relationships between classes:

Association: 

Directed relationship, signifying that one class creates objects of another, but not the other way around, e.g. class *Main* creates objects of class *Thing*.

Composition: 

A more specific version of Association, signifying that when a class creates an object of another class, that object is destroyed when the "container" class is destroyed. For example, when the *Main* class object is destroyed, all objects created within it are destroyed.

Aggregation: 

Another specific version of Association, specifying the opposite of Composition: if the "container" class is destroyed, objects it is associated with are not destroyed. For example, if a *Professor* object is destroyed, the *Course* objects associated with it still remain - the course does not cease to exist if the professor dies.

Inheritance: 

Used to signify when a child class inherits from a parent class.